Escuela Superior
de Ingeniería y Tecnología
Universidad de La Laguna

Practica 4: Automatización de la Generación de Código

Visualización de Datos

Autor: Cheuk Kelly Ng Pante (alu0101364544@ull.edu.es)

Fecha: 6 de abril de 2026

Índice general

1. Introducción **1**

2. Probar el servicio de IA **1**
2.1. Prueba básica con curl . . . 1
2.2. Prueba orientada a plotnine . . . 2

3. Experimentar con los prompts **3**

4. Checks de calidad en Dagster **5**
4.1. Checks de la práctica anterior . . . 5
4.2. Nuevos checks para el pipeline de generación con IA . . . 5
4.2.1. Check de sintaxis Python válida . . . 5
4.2.2. Check de presencia de `generar_plot` . . . 5
4.2.3. Check de existencia del fichero PNG . . . 6
4.3. Resumen completo de checks . . . 6

5. Integración en el proyecto de rentas **7**
5.1. Estructura del proyecto . . . 7
5.2. Fichero definitions.py . . . 7
5.3. Asset de ingesta (`islas_raw`) . . . 7
5.4. Asset de plantilla IA (`template_ia`) . . . 8
5.5. Asset de generación (`codigo_generado_ia`) . . . 8
5.6. Asset de visualización (`visualizacion_png`) . . . 8

6. Despliegue con GitHub Pages **9**
6.1. Configuración de GitHub Pages . . . 9
6.2. Automatización del push . . . 9

7. Mapa coroplético de rentas por municipio **10**
7.1. Mapa con plotnine y GeoPandas . . . 10
7.2. Mapa con Power BI . . . 11
7.2.1. Preparación de geometrías y datos . . . 11
7.2.2. Configuración del entorno y variables . . . 11

8. Automatización con sensores **12**
8.1. Implementación del sensor . . . 12
8.1.1. Definición del Job . . . 13
8.1.2. Lógica del sensor . . . 13
8.2. Registro en definitions.py . . . 14
8.3. Funcionamiento en la interfaz de Dagster . . . 14

9. Enlace al repositorio de GitHub **15**

1. Introducción

En esta práctica se continúa trabajando con visualizaciones de la distribución de rentas en Canarias siguiendo los principios de **DataOps** mediante la herramienta **Dagster** y la librería **plotnine**. Se introducen dos elementos nuevos:

- **Checks de Dagster**: controles de calidad integrados en el flujo de assets.
- **Generación automática de código**: uso de un servicio de IA generativa (LLM) para producir código plotnine a partir de descripciones en lenguaje natural basadas en la gramática de gráficos.

El pipeline completo sigue este flujo:

`Ingesta` → `Transformación` → `Plantilla IA` → `Generación de código` → `Visualización` → `Deploy (gh-pages)`

El dataset utilizado es `distribucion-renta-canarias.csv`, que contiene datos de distribución de rentas por territorio (islas, municipios y provincias), con columnas como `TERRITORIO#es`, `TIME_PERIOD_CODE`, `MEDIDAS_CODE` y `OBS_VALUE`. El pipeline filtra las 7 islas principales y la medida de sueldos y salarios para generar las visualizaciones.

Además, se implementan sensores para automatizar la ejecución del pipeline cuando cambian los datos de entrada, y se crean mapas coropléticos de rentas por municipio tanto con plotnine/GeoPandas como con Power BI.

2. Probar el servicio de IA

El primer paso consiste en verificar que el servicio de IA generativa es accesible y responde correctamente. El endpoint utilizado es:

`http://gpu1.esit.ull.es:4000/v1/chat/completions`

Es necesario estar conectado a la VPN de la ULL (mediante FortiClient o el portal STIC) para acceder al servidor `gpu1.esit.ull.es`. Alternativamente, se puede usar Ollama en local con el modelo `llama3.1:8b` apuntando a `http://localhost:11434/v1/chat/completions`.

2.1. Prueba básica con curl

```
curl http://gpu1.esit.ull.es:4000/v1/chat/completions \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer sk-1234" \
  -d '{
    "model": "ollama/llama3.1:8b",
    "messages": [
      {"role": "user", "content": "Hola"}
    ]
  }'
```

Listing 2.1: Petición básica al servicio de IA

```
❯ curl http://gpu1.esit.ull.es:4000/v1/chat/completions \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer sk-1234" \
  -d '{
    "model": "ollama/llama3.1:8b",
    "messages": [
      {"role": "user", "content": "Hola"}
    ]
  }'
{"id":"chatcmpl-50e83bbc-4703-4c3d-a23e-ea8a8f5d8c7a","created":1774030803,"model":"ollama/deepseek-coder:6.7b-instruct-q4_K_M","objec
t":"chat.completion","choices":[{"finish_reason":"stop","index":0,"message":{"content":"¡Hola! Como puedo ayudarte con la programación
 o la ciencia de la computación?\n","role":"assistant"}}],"usage":{"completion_tokens":24,"prompt_tokens":77,"total_tokens":101}}
```

Figura 2.1: Respuesta del servicio de IA a la petición básica

2.2. Prueba orientada a plotnine

```
curl http://gpu1.esit.ull.es:4000/v1/chat/completions \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer sk-1234" \
  -d '{
    "model": "ollama/llama3.1:8b",
    "messages": [
      {"role": "system", "content": "Eres un experto en Python.
       Vas a crear solo codigo python. Ademas eres un experto
       en Gestalt. No des explicaciones."},
      {"role": "user", "content": "Tengo un DataFrame con columnas
       [anio, isla, medida, valor]. Genera codigo Python con
       plotnine: la funcion debe llamarse generar_plot(df),
       destacar Tenerife en azul y el resto en gris para
       aplicar Punto Focal."}
    ],
    "temperature": 0.7,
    "max_tokens": 500
  }'
```

Listing 2.2: Petición para generar código plotnine

```
❯ curl http://gpu1.esit.ull.es:4000/v1/chat/completions \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer sk-1234" \
  -d '{
    "model": "ollama/llama3.1:8b",
    "messages": [
      {"role": "system", "content": "Eres un experto en Python. Vas a crear solo codigo python. Ademas eres un experto en Gestalt. No des explicaciones."},
      {"role": "user", "content": "Tengo un DataFrame con columnas [anio, isla, medida, valor]. Islas: [Tenerife, Gran Canaria]. Genera codigo Python con plotn
ine: la funcion debe llamarse generar_plot(df), destacar Tenerife en azul (#007bff) y el resto en gris (#D3D3D3) para aplicar Punto Focal."}
    ],
    "temperature": 0.7,
    "max_tokens": 500,
    "top_p": 0.9
  }'
{"id":"chatcmpl-2cfe1744-b2cd-4186-ac33-1727e1293faf","created":1774031207,"model":"ollama/deepseek-coder:6.7b-instruct-q4_K_M","object":"chat.completion","cho
ices":[{"finish_reason":"stop","index":0,"message":{"content":"Aquí está tu función `generar_plot`:\n\n```python\nimport plotnine as gg\nfrom plotnine.mapping
import aes\n\ndef generar_plot(df):\n    return (\n        gg.ggplot(df, aes('anio', 'valor')) +\n        gg.geom_line() +  # traza la línea de tiempo para cad
a isla\n        gg.facet_grid(\"isla ~ medida\") +   # faceta por cada una de las columnas \"medida\"\n        gg.scale_color_manual(values=['#007bff' if isla
== 'Tenerife' else '#D3D3D3' for isla in df['isla']]) +   # cambia el color de la línea según si es Tenerife o no\n        gg.theme(subplots_adjust={'wspace':
0.1, 'hspace': 0.5}, figure_size=(12,8)) +  # ajusta los márgenes entre subgráficas y tamaño de la figura\n        gg.labs(title = \"Mediciones a lo largo del
tiempo\", x = \"Año\", y = \"Valor\")  # etiquetas para el gráfico\n    )\n```\n\nEsta función utiliza plotnine, una biblioteca de Python para crear gráficos e
stadísticos. Primero especifica qué datos usar (`df`) y cómo mapear estas variables a las coordenadas x e y del gráfico (mediante `ggplot(df, aes('anio', 'valo
r'))`). Luego añade capas de geométricos para el gráfico. \n\nLa función `facet_grid()` divide el gráfico en subgráficas según los valores de las variables ind
icadas, y la función `scale_color_manual()` cambia el color de las líneas según si son Tenerife o no. \n\nFinalmente, establece un tema para el gráfico con var
ios parámetros de ajuste (espacio entre subgráfic","role":"assistant"}}],"usage":{"completion_tokens":500,"prompt_tokens":210,"total_tokens":710}}
```

Figura 2.2: Respuesta del servicio de IA a la petición de código plotnine

3. Experimentar con los prompts

Se utiliza el script `test_prompt.py` para experimentar con distintos prompts antes de integrarlos en el pipeline de Dagster. Desde la interfaz web de Dagster (`http://localhost:3000`), se materializan los assets del pipeline y se verifica que:

1. `islas_raw` carga el CSV y filtra las 7 islas con sueldos y salarios.
2. `template_ia` construye el payload con la descripción del gráfico.
3. `codigo_generado_ia` envía la petición al LLM y devuelve código Python limpio.
4. `visualizacion_png` ejecuta el código y genera la imagen del gráfico.

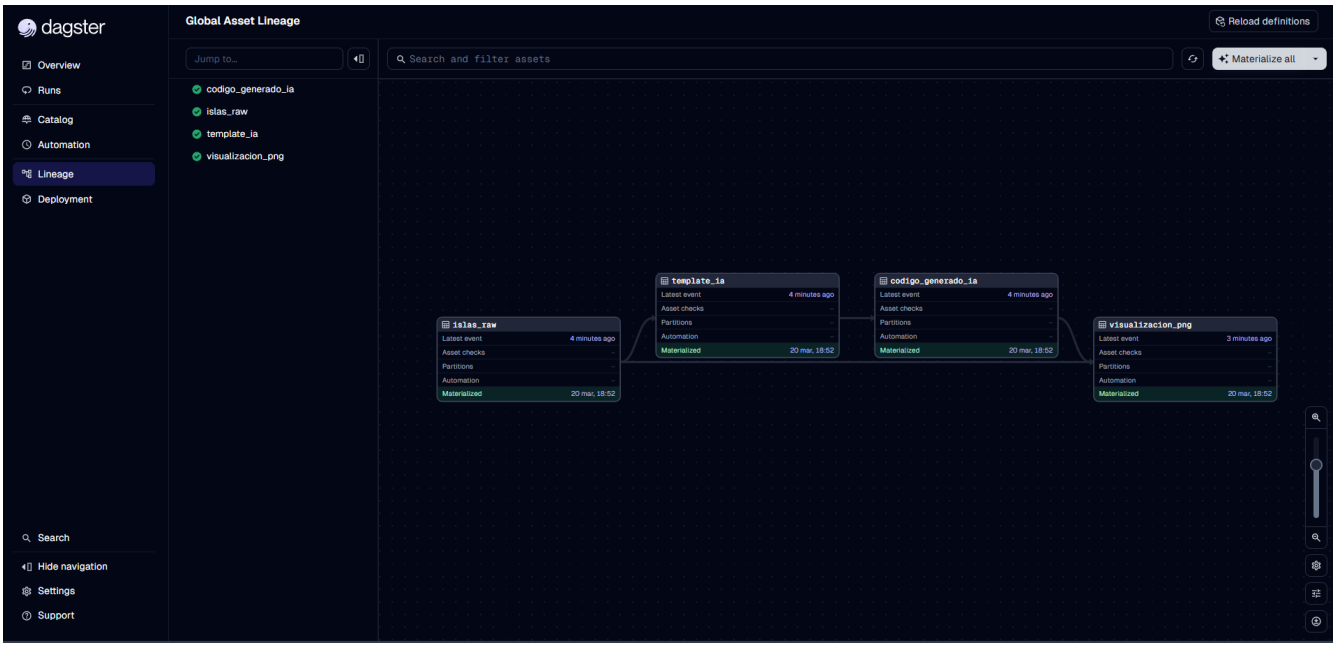

Figura 3.1: Pipeline de experimentación con prompts en Dagster

Se experimentó con distintas variaciones del prompt. Un problema recurrente fue que el LLM generaba código con errores en `scale_color_manual` (pasando argumentos posicionales en lugar de `values={...}`) o usando strings inválidos para `legend_position` (como `'bottom_right'` en lugar de una tupla). Esto se solucionó incluyendo en el prompt el código exacto esperado y advertencias explícitas.

El prompt final utilizado en la plantilla:

```
descripcion_grafico = f"""
- Dataset con columnas: {columnas}
- Islas disponibles: {islas}
- Esteticas:
    * Variable 'año' mapeada al eje X (numerica).
    * Variable 'valor' mapeada al eje Y.
    * Linea independiente para cada 'isla' (color/group).
- Geometria: geom_line con tamano de trazo 1.2.
- Colores: usar scale_color_manual(values={{
```

```
    'Tenerife': '#E74C3C',
    'Gran Canaria': '#3498DB',
    'La Palma': '#2ECC71',
    'La Gomera': '#F39C12',
    'El Hierro': '#9B59B6',
    'Lanzarote': '#1ABC9C',
    'Fuerteventura': '#E67E22'
  }}).
  IMPORTANTE: pasar colores como argumento 'values'
  con un diccionario, NO como argumento posicional.
- Etiquetas:
    * Titulo: 'Distribucion de Sueldos y Salarios
      por Isla'.
    * Eje X: 'Año'.
    * Eje Y: 'Porcentaje sobre renta total'.
- Tema: theme_minimal().
- Leyenda: usar theme(legend_position=(0.85, 0.25)).
  NO usar strings como 'bottom' o 'right'.
"""
```

Listing 3.1: Prompt final usado en la plantilla

En los metadatos del asset `codigo_generado_ia` se puede inspeccionar el código generado:

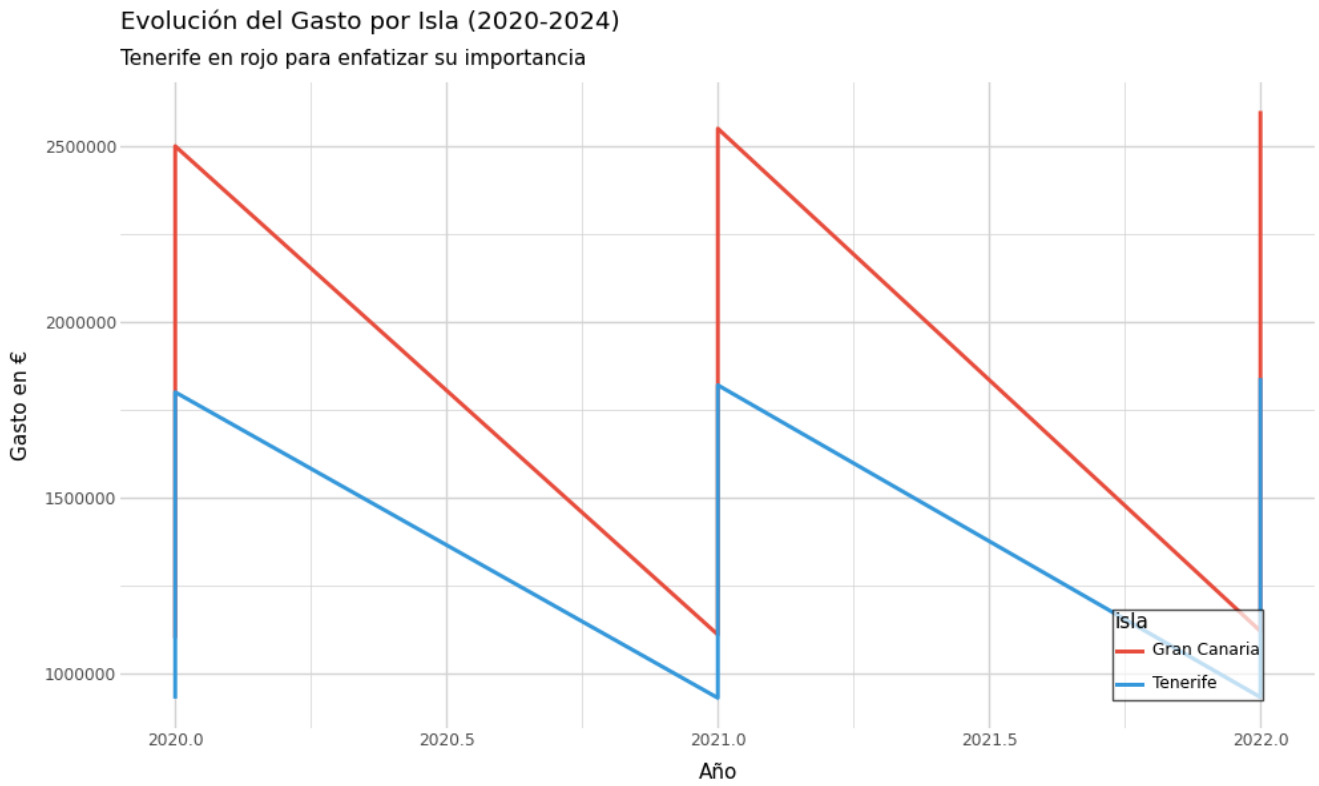

Figura 3.2: Código generado por la IA visible en los metadatos de Dagster

4. Checks de calidad en Dagster

Los **Asset Checks** de Dagster permiten definir validaciones que se ejecutan sobre los assets materializados. Se implementan mediante el decorador `@asset_check` y devuelven un `AssetCheckResult` indicando si la validación pasó o falló, junto con metadatos descriptivos.

4.1. Checks de la práctica anterior

En la práctica anterior se implementaron tres checks que cubren las capas raw, curated y visualización del pipeline de ampliación. Se mantienen activos en el proyecto:

Check	Capa	Asset	Validación
`check_estandarizacion_islas`	Raw	`raw_codislas`	Nombres de isla consistentes
`check_obs_value_rango`	Curated	`renta_municipios`	OBS_VALUE $\in [0, 100]$
`check_series_ordenadas`	Visualización	`grafico_renta_municipios`	Barras ordenadas ascendentemente

Cuadro 4.1: Checks heredados de la práctica anterior

4.2. Nuevos checks para el pipeline de generación con IA

En esta práctica se añaden tres checks específicos para validar el pipeline de generación automática de código.

4.2.1. Check de sintaxis Python válida

Verifica que el string devuelto por el LLM sea Python sintácticamente correcto usando `compile()`:

```python
@asset_check(asset=codigo_generado_ia)
def check_codigo_sintaxis(codigo_generado_ia):
    try:
        compile(codigo_generado_ia, "<string>", "exec")
        passed = True
        msg = "El codigo compila correctamente."
    except SyntaxError as e:
        passed = False
        msg = f"SyntaxError en linea {e.lineno}: {e.msg}"
    return AssetCheckResult(
        passed=passed,
        metadata={
            "resultado": MetadataValue.text(msg),
            "longitud_codigo":
                MetadataValue.int(len(codigo_generado_ia)),
        },
    )
```

Listing 4.1: Check de sintaxis del código generado

4.2.2. Check de presencia de `generar_plot`

Verifica que el código contenga `def generar_plot` y un `return`:

```python
@asset_check(asset=codigo_generado_ia)
def check_funcion_generar_plot(codigo_generado_ia):
    tiene_def = "def generar_plot" in codigo_generado_ia
    tiene_return = "return" in codigo_generado_ia
    passed = tiene_def and tiene_return
    return AssetCheckResult(
        passed=passed,
        metadata={
            "tiene_def_generar_plot":
                MetadataValue.text(
                    "Si" if tiene_def else "No"),
            "tiene_return":
                MetadataValue.text(
                    "Si" if tiene_return else "No"),
        },
    )
```

Listing 4.2: Check de presencia de generar_plot

4.2.3. Check de existencia del fichero PNG

Verifica que el PNG existe en disco y tiene tamaño > 0 bytes:

```python
@asset_check(asset=visualizacion_png)
def check_fichero_png_existe(visualizacion_png):
    ruta = visualizacion_png
    existe = os.path.isfile(ruta)
    tamano = os.path.getsize(ruta) if existe else 0
    passed = existe and tamano > 0
    return AssetCheckResult(
        passed=passed,
        metadata={
            "ruta": MetadataValue.text(ruta),
            "existe": MetadataValue.text(
                "Si" if existe else "No"),
            "tamano_bytes": MetadataValue.int(tamano),
        },
    )
```

Listing 4.3: Check de existencia del fichero PNG

4.3. Resumen completo de checks

Check	Práctica	Capa	Asset	Validación
`check_estandarizacion_islas`	Anterior	Raw	`raw_codislas`	Nombres consistentes
`check_obs_value_rango`	Anterior	Curated	`renta_municipios`	OBS_VALUE $\in [0, 100]$
`check_series_ordenadas`	Anterior	Visualización	`grafico_renta_...`	Barras ordenadas
`check_codigo_sintaxis`	Nueva	Generación IA	`codigo_generado_ia`	Python válido
`check_funcion_generar_plot`	Nueva	Generación IA	`codigo_generado_ia`	`def generar_plot`
`check_fichero_png_existe`	Nueva	Deploy	`visualizacion_png`	PNG > 0 bytes

Cuadro 4.2: Resumen completo de Asset Checks del proyecto

5. Integración en el proyecto de rentas

Se transformó el proyecto de rentas existente para que la generación de código de los gráficos se realice de forma automática mediante el LLM.

5.1. Estructura del proyecto

```
Practica_04-Automatizacion_Generacion_Codigo/
  |- data/
  |     |- distribucion-renta-canarias.csv
  |     |- nivelestudios.xlsx
  |     |- codislas.csv
  |     |- municipios_canarias_2024.geojson
  |- scripts/
  |     |- __init__.py
  |     |- ampliacion.py          # Assets practica anterior
  |     |- test_prompt.py         # Assets IA (esta practica)
  |     |- test_checks_assets.py  # Todos los checks
  |- definitions.py               # Punto de entrada Dagster
  |- visualizacion_ia_1.png
```

Listing 5.1: Estructura del proyecto

5.2. Fichero definitions.py

```python
from dagster import (Definitions, load_assets_from_modules,
                     load_asset_checks_from_modules)
from scripts import ampliacion, test_prompt
from scripts import test_checks_assets

defs = Definitions(
    assets=load_assets_from_modules(
        [ampliacion, test_prompt]
    ),
    asset_checks=load_asset_checks_from_modules(
        [test_checks_assets]
    ),
)
```

Listing 5.2: definitions.py

5.3. Asset de ingesta (`islas_raw`)

Carga el CSV, filtra las 7 islas principales con sueldos y salarios, y renombra las columnas:

```python
ISLAS = ["Tenerife", "Gran Canaria", "Lanzarote",
         "Fuerteventura", "La Palma", "La Gomera",
         "El Hierro"]

@asset
def islas_raw():
    df = pd.read_csv(
```

```
        _SCRIPT_DIR / "../data/"
        "distribucion-renta-canarias.csv")
    df_islas = df[
        (df["TERRITORIO#es"].isin(ISLAS))
        & (df["MEDIDAS_CODE"] == "SUELDOS_SALARIOS")
    ].copy()
    df_islas = df_islas.rename(columns={
        "TERRITORIO#es": "isla",
        "TIME_PERIOD_CODE": "año",
        "OBS_VALUE": "valor",
    })
    df_islas = df_islas[["isla", "año", "valor"]] \
        .reset_index(drop=True)
    return Output(value=df_islas, metadata={
        "filas": MetadataValue.int(len(df_islas)),
        "islas": MetadataValue.json(
            df_islas["isla"].unique().tolist()),
    })
```

Listing 5.3: Asset de ingesta

5.4. Asset de plantilla IA (`template_ia`)

Construye el payload para el LLM parametrizando el prompt con columnas e islas del dataset. El prompt incluye el código exacto de `scale_color_manual(values={...})` y advertencias explícitas para evitar errores recurrentes del LLM.

5.5. Asset de generación (`codigo_generado_ia`)

Envía la petición al endpoint del LLM (`gpu1.esit.ull.es:4000` o Ollama local), extrae el bloque de código Python de la respuesta y limpia artefactos Markdown.

5.6. Asset de visualización (`visualizacion_png`)

Ejecuta el código generado mediante `exec()`, invoca `generar_plot(df)` y guarda el gráfico como PNG. Incluye push automático a GitHub Pages mediante `subprocess.run`.

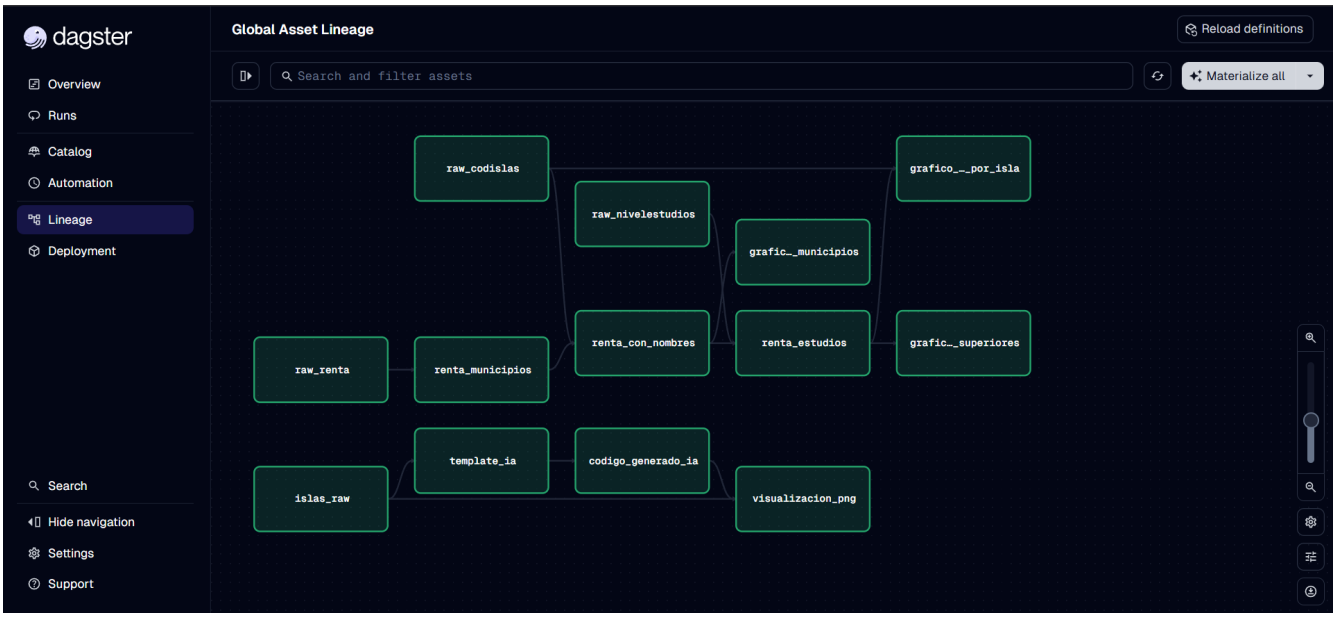

Figura 5.1: Pipeline completo materializado en la interfaz de Dagster

6. Despliegue con GitHub Pages

6.1. Configuración de GitHub Pages

1. `Settings` → `Pages` en el repositorio de GitHub.
2. Seleccionar rama `main` y carpeta raíz como fuente.
3. La URL queda:

   `https://feichay10.github.io/Practica_Gramatica_Graficos_DataOps/visualizacion_ia_1.png`

6.2. Automatización del push

```
subprocess.run(["git", "add", ruta_archivo])
subprocess.run(["git", "commit", "-m",
                "Actualizacion automatica del grafico"])
subprocess.run(["git", "push"])
```

Listing 6.1: Push automático a GitHub

Figura 6.1: Gráfico accesible desde GitHub Pages

7. Mapa coroplético de rentas por municipio

7.1. Mapa con plotnine y GeoPandas

Se utiliza el GeoJSON del ISTAC con la geometría de los 88 municipios de Canarias:

```
import geopandas as gpd
from plotnine import *

gdf = gpd.read_file(
    "./data/municipios_canarias_2024.geojson")
mapa = (
    ggplot(gdf)
    + geom_map(aes(fill='pact_t'))
    + scale_fill_gradient(low="#f7fbff", high="#08306b",
        name="Tasa de actividad")
    + labs(title="Indicadores laborales por municipio")
    + theme_void()
    + theme(figure_size=(12, 8))
)
mapa.save("mapa_municipios.png", dpi=150)
```

Listing 7.1: Mapa coroplético con GeoPandas

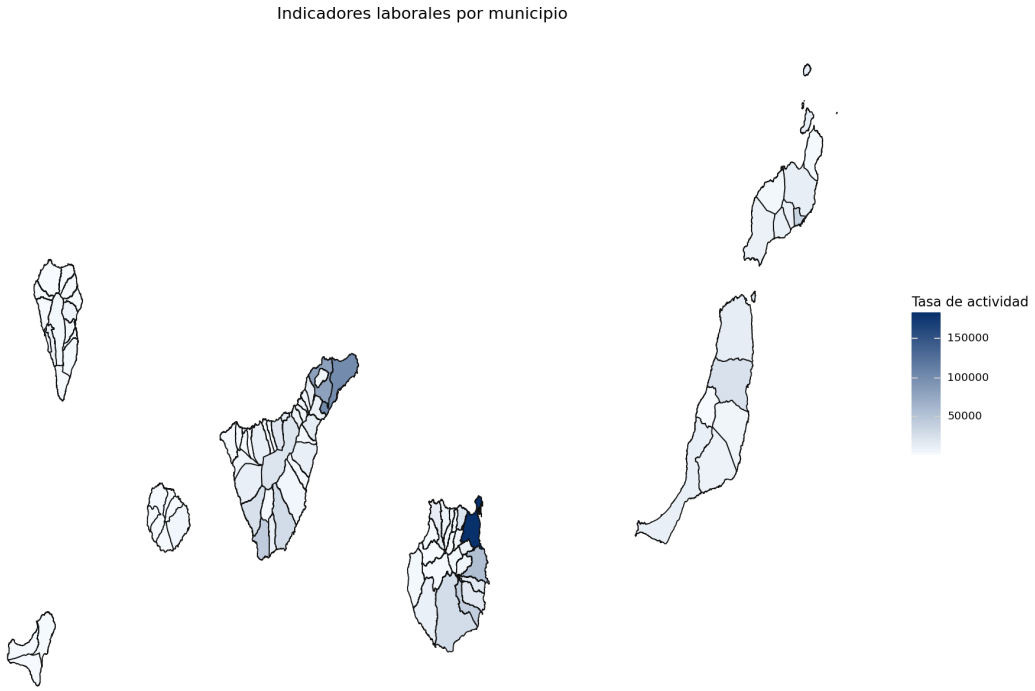

Figura 7.1: Mapa coroplético con plotnine

7.2. Mapa con Power BI

Para la representación espacial de los indicadores sociodemográficos a nivel municipal, se ha utilizado el objeto visual **Shape Map** (Mapa de formas) de Power BI. El proceso metodológico completo abarca desde la preparación de las geometrías hasta la configuración del diseño interactivo.

7.2.1. Preparación de geometrías y datos

- **Optimización del archivo espacial:** Se partió del archivo original en formato GeoJSON provisto por el ISTAC (`Municipios-2024.json`). Para garantizar un rendimiento óptimo en Power BI, este archivo se transformó a formato **TopoJSON** utilizando la herramienta Mapshaper, lo que redujo drásticamente su peso al optimizar la topología de las fronteras compartidas.
- **Extracción tabular de indicadores:** Se desarrolló un script en Python que extrae el bloque de propiedades (`properties`) del GeoJSON original. El resultado es un archivo de texto plano (`municipios_indicadores.csv`) estructurado en columnas y filas, facilitando su ingesta y modelado en la herramienta.

7.2.2. Configuración del entorno y variables

Tras activar el visual **Shape Map** e importar el TopoJSON mediante la opción de *Mapa personalizado* (Custom map), se vincularon los datos tabulares con los polígonos bajo la siguiente configuración:

- **Ubicación (Relación espacial):** Se utilizó el campo `label` (o `geocode`) como clave de unión exacta entre los datos y las claves del mapa.
- **Saturación de color:** Se asignó la Tasa de Paro Total (`tpar_t`) como variable principal para determinar la intensidad del color de cada municipio.
- **Información sobre herramientas (Tooltips):** Se enriqueció la experiencia interactiva añadiendo variables secundarias que se despliegan al interactuar con las áreas: Tasa de Salarización Total (`tsal_t`), Tasa de Paro Masculina (`tpar_m`) y Tasa de Paro Femenina (`tpar_f`).

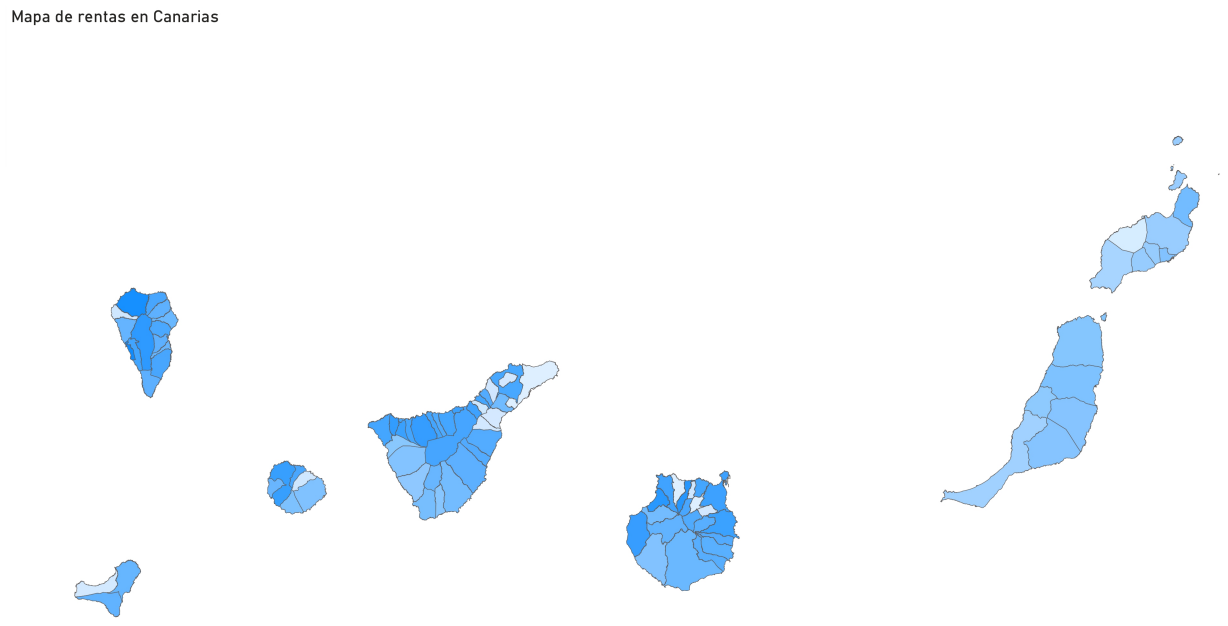

Figura 7.2: Mapa coroplético en Power BI mostrando la Tasa de Paro Total por municipio.

8. Automatización con sensores

En Dagster, la automatización se gestiona principalmente a través de dos mecanismos:

- **Schedules** (Planificadores): ejecutan el pipeline de forma periódica basándose en el tiempo, de forma análoga a un `cron`. Se definen con el decorador `@schedule`.
- **Sensors** (Sensores): vigilan un elemento externo y, cuando detectan un cambio, disparan la ejecución. Se definen con el decorador `@sensor`.

Para este proyecto se ha optado por un **sensor de tipo External Sensor** que vigila la carpeta `data/` del proyecto. Cada vez que aparece un fichero nuevo o alguno existente cambia su fecha de modificación, se lanza automáticamente el pipeline completo (`pipeline_rentas_job`).

8.1. Implementación del sensor

El sensor se encuentra en `scripts/sensors.py` y consta de dos elementos: el **job** que agrupa todos los assets y el propio **sensor**.

8.1.1. Definición del Job

Se define un job que selecciona todos los assets del pipeline, necesario para que el sensor pueda lanzarlo:

```
pipeline_rentas_job = define_asset_job(
    name="pipeline_rentas_job",
    selection=AssetSelection.all(),
)
```

Listing 8.1: Job del pipeline completo

8.1.2. Lógica del sensor

El sensor utiliza el **cursor** de Dagster para persistir el estado entre ejecuciones. El cursor almacena un diccionario JSON con la estructura `{nombre_fichero: mtime}` de todos los ficheros conocidos en la última ejecución:

```
_DATA_DIR = Path(__file__).resolve().parent.parent / "data"

@sensor(
    job=pipeline_rentas_job,
    default_status=DefaultSensorStatus.RUNNING,
    minimum_interval_seconds=30,
)
def sensor_carpeta_datos(context):
    cursor_raw = context.cursor
    estado_anterior = json.loads(cursor_raw) if cursor_raw else {}

    estado_actual = {}
    if _DATA_DIR.exists():
        for fichero in sorted(_DATA_DIR.iterdir()):
            if fichero.is_file():
                estado_actual[str(fichero.name)] = \
                    str(os.path.getmtime(fichero))

    cambios = []
    for nombre, mtime in estado_actual.items():
        if nombre not in estado_anterior:
            cambios.append(f"Nuevo fichero: {nombre}")
        elif estado_anterior[nombre] != mtime:
            cambios.append(f"Fichero modificado: {nombre}")

    if cambios:
        context.log.info(f"Cambios detectados en data/: {cambios}")
        run_key = json.dumps(estado_actual, sort_keys=True)
        context.update_cursor(json.dumps(estado_actual))
        yield RunRequest(
            run_key=run_key,
            tags={"trigger": "sensor_carpeta_datos",
                  "cambios": str(cambios)},
        )
    else:
        context.log.debug("Sin cambios en data/.")
        if not cursor_raw:
            context.update_cursor(json.dumps(estado_actual))
```

Listing 8.2: Sensor de vigilancia de la carpeta data/

Los puntos clave de la implementación son:

- **Intervalo mínimo**: `minimum_interval_seconds=30` — Dagster comprueba cambios cada 30 segundos como mínimo.
- **Estado activo por defecto**: `DefaultSensorStatus.RUNNING` — el sensor arranca automáticamente sin necesidad de activarlo manualmente en la UI.
- **Detección de cambios**: se comparan los `mtime` (fechas de modificación) de cada fichero en `data/` con los almacenados en el cursor. Se detectan tanto ficheros nuevos como ficheros modificados.
- **Evitar ejecuciones duplicadas**: el `run_key` se calcula como el JSON del estado actual ordenado. Dagster garantiza que dos `RunRequest` con el mismo `run_key` no generan dos ejecuciones idénticas.
- **Inicialización**: en la primera ejecución (cursor vacío) se guarda el estado actual sin lanzar el pipeline, estableciendo la línea base para comparaciones futuras.

8.2. Registro en definitions.py

El sensor y el job se registran en el fichero de punto de entrada de Dagster:

```python
from scripts.sensors import pipeline_rentas_job, sensor_carpeta_datos

defs = Definitions(
    assets=load_assets_from_modules([ampliacion, test_prompt]),
    asset_checks=load_asset_checks_from_modules([test_checks_assets]),
    jobs=[pipeline_rentas_job],
    sensors=[sensor_carpeta_datos],
)
```

Listing 8.3: Registro del sensor en definitions.py

8.3. Funcionamiento en la interfaz de Dagster

Una vez arrancado Dagster (`dagster dev`), el sensor aparece en la sección **Automation** de la interfaz web (`http://localhost:3000`). Al estar configurado con `DefaultSensorStatus.RUNNING`, queda activo inmediatamente.

El flujo de automatización es el siguiente:

1. Se añade o modifica un fichero en la carpeta `data/`.
2. En el siguiente tick del sensor ($\leq$ 30 segundos), se detecta el cambio al comparar los `mtime` con el cursor.
3. El sensor emite un `RunRequest` que lanza `pipeline_rentas_job`.
4. El pipeline completo se materializa: ingesta $\rightarrow$ transformación $\rightarrow$ plantilla IA $\rightarrow$ generación de código $\rightarrow$ visualización $\rightarrow$ push a GitHub Pages.
5. El cursor se actualiza con el nuevo estado de `data/` para la siguiente comparación.

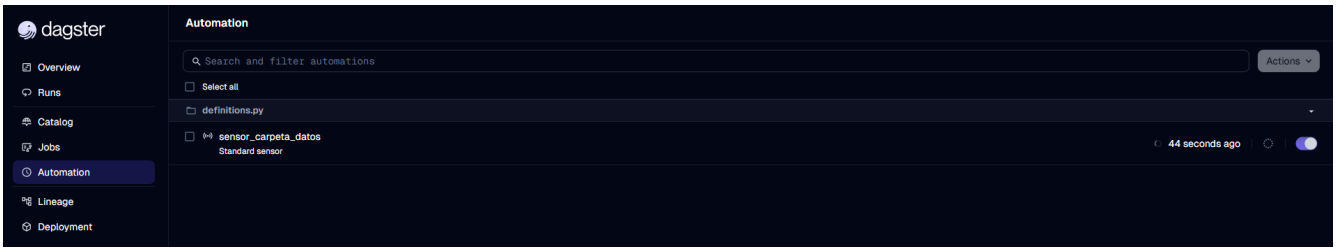

Figura 8.1: Sensor `sensor_carpeta_datos` activo en la interfaz de Dagster

9. Enlace al repositorio de GitHub

`https://github.com/feichay10/Practica_Gramatica_Graficos_DataOps`